

Sharing at a Cost: Engagement Maximization via Forwarding Control on Social Cascades

Xiaolong Chen

The Hong Kong University of Science and Technology
(Guangzhou)

Nansha, Guangzhou, China
xchen738@connect.hkust-gz.edu.cn

Jing Tang

The Hong Kong University of Science and Technology
(Guangzhou)

The Hong Kong University of Science and Technology
jingtang@ust.hk

ABSTRACT

Social sharing drives a large fraction of content consumption on modern immersive-feed platforms. Yet the act of forwarding carries a hidden cost that classical Influence Maximization overlooks. When a user pauses the feed to forward it to others, the platform loses session engagement it would otherwise have collected. This paper studies *Engagement Maximization via Forwarding Control* (EMFC), in which the platform deliberately adds friction to users' forwarding actions in order to increase total engagement. We model this as a continuous optimization over per-user forwarding propensities on a social graph under the Independent Cascade model, where each user carries a session value and a friction coefficient and the platform chooses how strongly to discourage each user's forwarding. We first show that the problem admits no constant-factor approximation unless $P = NP$, which rules out the approximation guarantees of classical submodular IM. Despite this hardness, we prove that the objective is multilinear and *bang-bang*, so the continuous problem collapses without loss to a per-user binary decision, making a coordinate-local optimum attainable.

A natural approach is coordinate ascent over the vertices. However, computing the objective function is $\#P$ -hard, and existing estimation methods are either inefficient or require redrawing samples at each iteration, which prevents convergence. To address these issues, we introduce the concept of *gated reverse-reachable graph* (GRRG), whose samples are drawn once, frozen, and reused across the entire search, providing an efficient unbiased estimate of engagement at any control vector. Combined with coordinate ascent framework, it yields an approximate coordinate-local optimum. To further improve efficiency, we decompose the gradient derive control-independent bounds that let us lock users whose optimal endpoint is already settled before the sweep begins. The proposed pruning technique brings an acceleration of up to 60%. Experiments on 10 real-world datasets evaluate the effectiveness, efficiency, and sensitivity of the proposed methods. The results show improved engagement over natural heuristics, reduced runtime compared to a Monte Carlo solver at comparable solution quality, and scalability to million-size graphs.

PVLDB Reference Format:

Xiaolong Chen and Jing Tang. Sharing at a Cost: Engagement Maximization via Forwarding Control on Social Cascades. PVLDB, 20(1): XXX-XXX, 2027. doi:XX.XX/XXX.XX

This work is licensed under the Creative Commons BY-NC-ND 4.0 International License. Visit <https://creativecommons.org/licenses/by-nc-nd/4.0/> to view a copy of this license. For any use beyond those covered by this license, obtain permission by emailing info@vldb.org. Copyright is held by the owner/author(s). Publication rights licensed to the VLDB Endowment.

Proceedings of the VLDB Endowment, Vol. 20, No. 1 ISSN 2150-8097.

PVLDB Artifact Availability:

The source code, data, and/or other artifacts have been made available at <https://github.com/chenxlong3/EMFC>.

1 INTRODUCTION

Modern content platforms (TikTok, WeChat Channels, Instagram Reels) are dominated by *immersive feeds*: a user opens the app, consumes algorithmically ranked items in a sequential session, and occasionally *forwards* an item to specific friends. Forwarding turns a single ranked impression into a chain of additional impressions that propagate along the social network, and is widely regarded as one of the strongest organic-growth signals on such platforms.

A central operational question is which users the platform should incentivize to forward content or suppress from doing so, in order to maximize platform-level engagement. This question naturally connects to the Influence Maximization (IM) literature initiated by Kempe et al. [16], where a planner selects a small seed set S_0 in a social graph to maximize the expected number of activated nodes under a stochastic diffusion model such as the Independent Cascade (IC) or Linear Threshold (LT) model.

The conventional IM problem assumes that more diffusion is always better and every additional activation contributes to the objective at no internal cost. This assumption holds in the original epidemiology- and viral-marketing-inspired settings [8, 21, 30], where the spread of a product or information is purely beneficial to the planner. However, it breaks down on modern immersive-feed platforms. Forwarding is a deliberate action: choosing a recipient, composing a message, or confirming a share dialog all require the forwarder u to leave the immersive feed and interact with a separate UI surface. During this interruption, u is no longer consuming feed content, and the platform loses session time that would otherwise have been generated. Empirically, an explicit forwarding event correlates with a measurable drop in u 's same-session engagement, and the size of this drop varies systematically across users. Classical IM does not capture this internal cost, and as a result optimizing for raw cascade reach can in fact *reduce* platform-level engagement when the friction dominates. We refer to this displaced engagement as *forwarding friction*, and to the dual nature of forwarding, which reaches social gain but loses engagement on the platform side.

To maximize total engagement under forwarding friction, we cast the problem as a continuous optimization over per-user forwarding propensities. Each user carries a control variable that encodes how strongly the platform encourages that user to forward, through ranking, UI prompts, or incentive payments [12], together with a session value that captures the engagement generated when the

doi:XX.XX/XXX.XX

user consumes the content and a friction coefficient that captures the engagement lost each time the user forwards. The expected engagement under a given control vector combines these three quantities with the activation probability of each user, and the platform maximizes this quantity over the box of admissible controls. We study the structure of this objective and show that it is affine in each individual control, which makes the objective *bang-bang*. An optimal solution therefore always sits at a corner of the box, so the continuous problem collapses without loss to a per-user binary decision of whether to encourage or suppress forwarding. Despite this simplification the problem remains hard. We prove that it admits no constant-factor approximation in the worst case unless $P = NP$, which rules out constant approximation guarantees.

The critical algorithmic challenge is the flip test itself. Deciding whether flipping a given user raises or lowers engagement requires the marginal effect of that user on the objective, and this marginal depends on activation probabilities under the cascade, which are $\#P$ -hard to evaluate exactly and must be estimated by sampling. Drawing fresh samples for every candidate flip is both expensive and unreliable. It repeats the sampling procedure once per flip, and because the two sides of each comparison come from independent draws, their errors accumulate rather than cancel, and the resulting noise easily obscures small marginals, preventing convergence.

Our main methodological contribution addresses this directly. We build a sampling structure, the *gated reverse-reachable graph* (GRRG) once and reuse it across the entire search. Each GRRG freezes the reverse-reachable subgraph rooted at a value-weighted node, together with the per-node gate thresholds, so that the contribution of every sample to the total engagement can be evaluated under *any* control vector without bias. Running coordinate ascent on this estimator yields an approximate coordinate-local optimum.

Even with this efficient estimator, sweeping over all users repeatedly is the dominant cost on large graphs, and much of that work is spent on users whose decision is a foregone conclusion. To accelerate this procedure, we propose a pruning technique that removes them before the search begins. We decompose each user’s gradient into a downstream-gain term and a self-friction term. By designing non-trivial bounds on both terms that hold regardless of the other controls, any user whose sign is already settled by these bounds is locked to its optimal corner and never enters the sweep. On the tested datasets, this removes over 60% of the decision variables while preserving the approximate coordinate-local optimality. Extensive experiments are conducted on 10 real-world datasets to validate the effectiveness and efficiency of the proposed algorithms. The results show that they yield higher engagement than natural heuristics, run faster than a Monte Carlo solver by orders of magnitude while achieving comparable solution quality, and scale to million-size graphs.

Contributions and organization. We summarize our contributions as follows.

- (1) **A new problem (§3).** We formulate and thoroughly analyze the *engagement maximization via forwarding control* (EMFC) problem, which aims to maximize total engagement on an immersive feed platform by modulating each user’s forwarding propensity.

Table 1: Frequently used notations

Notation	Description
$G = (V, E)$	A social network with node set V and edge set E
n, m	The number of nodes and edges in G
$p_{u,v}$	The transmission probability of edge (u, v)
q_v	The probability that the platform delivers the content to v directly
τ_v	The session value generated when v consumes the content
λ_v	The fraction of τ_v lost when v forwards
T	The total session value $\sum_{v \in V} \tau_v$
ξ_{lo}	The lowest admissible value of forwarding propensity
$\xi, \xi^{u \leftarrow a}$	A forwarding propensity vector, and the vector with ξ_u set to a while others remain unchanged
$A(\xi)$	The activated node set under ξ
$J(\xi)$	The expected total engagement.
$\Delta_u(\xi)$	The gain of switching ξ_u to the other endpoint
$R, \mathcal{R}$	A random GRRG, a collection of GRRG samples
$\Lambda_{\mathcal{R}}(\xi)$	The estimator function with random GRRGs

- (2) **Estimation via GRRG and certified approximate local optimality. (§4.3).** We introduce an efficient estimator via *gated reverse reachable graphs*. Combined with coordinate ascent, it certifies an approximate coordinate-local optimum.
- (3) **Pruning by gradient decomposition (§4.4).** By decomposing each user’s gradient, we derive control-independent bounds for the gradients. Users whose optimal endpoint is certain are locked before the search begins, reducing total runtime by up to 60%, without any loss in solution quality.
- (4) **Empirical validation (§6).** We validate the effectiveness and efficiency of our methods through experiments on 10 real-world datasets. The results show higher engagement than heuristics and orders-of-magnitude speedup over a Monte Carlo solver.

Missing proofs are provided in our technical report¹.

2 PRELIMINARIES

This section reviews some preliminaries that underlie our problem, which are standard in the influence maximization literature.

2.1 Social Graph and Cascade Model

Social graph. Let $G = (V, E)$ be a directed social graph with $n = |V|$ users and $m = |E|$ directed edges. An edge $(u, v) \in E$ indicates that user u can deliver a forwarded item to user v . Each edge is labeled with a transmission probability $p_{u,v} \in [0, 1]$, the probability that a forward from u along (u, v) causes v to consume the content, conditional on u having forwarded.

Independent Cascade (IC) model. A cascade proceeds in discrete time steps. At time 0, a set $\mathcal{S} \subseteq V$ of *seed* users is activated. At each subsequent step, every newly activated user u gets one chance to activate each currently inactive out-neighbor v independently with probability $p_{u,v}$. The process terminates when no new activations occur.

¹<https://github.com/chenxlong3/EMFC>

Live-edge representation. The IC model has an equivalent sampling formulation that is central to our algorithms. For each edge $(u, v) \in E$, independently retain the edge in a random subgraph H with probability $p_{u,v}$. Then $A(S)$ is exactly the set of nodes reachable from S in H . This representation converts a stochastic cascade into a deterministic reachability problem on a random graph.

2.2 Reverse Reachable Sets

In the classic influence maximization problem, computing exact influence spread is #P-hard. As a result, most existing approaches [1, 11, 16, 18, 25–27] rely on sampling to obtain unbiased estimates. State-of-the-art methods adopt a technique called *reverse influence sampling* (RIS), which is built on the concept of *random reverse reachable sets* (RR-sets). A random RR-set R is generated in two steps: first pick a node $v \in V$ uniformly at random, then sample a set R of nodes that can reach v , such that for any node $u \in V$, the probability that u appears in R equals the probability that u activates v . Intuitively, a seed set S whose nodes frequently show up in such RR-sets is likely to have high influence.

Formally, Borgs et al. [1] proved that $\sigma(S) = n \cdot \Pr[R \cap S \neq \emptyset]$. For convenience, we say that S covers R whenever $S \cap R \neq \emptyset$. Given a collection $\mathcal{R}$ of random RR-sets, the quantity $n \cdot \frac{|\mathcal{R}_S|}{|\mathcal{R}|}$ gives an unbiased estimator of $\sigma(S)$, where $\mathcal{R}_S$ denotes the subcollection of RR-sets covered by S . Under the RIS framework, after sampling enough RR-sets, existing algorithms essentially solve a maximum coverage problem over $\mathcal{R}$.

3 PROBLEM FORMULATION

This section formalizes the setting introduced in Section 1, building on the graph and cascade primitives of Section 2. We first describe the per-user attributes and the platform’s control (Section 3.1), then state the optimization problem, which we call *Engagement Maximization via Forward Control* (EMFC) (Section 3.2). We then establish the structural properties that our algorithms rely on and the hardness result that motivates us to attain a coordinate-local optimal solution (Section 3.3).

3.1 Problem Setup

Let $G = (V, E)$ be the social graph of Section 2, with edge transmission probabilities $\{p_{u,v}\}$. Every user $v \in V$ carries three scalars known to the platform. The *seed probability* $q_v \in [0, 1]$ is the chance that a user v spontaneously consumes the feeds in the platform. The *session value* $\tau_v \geq 0$ is the engagement, such as session length or watch time, that the platform realizes when v consumes the content. The *friction coefficient* $\lambda_v \in (0, 1)$ is the fraction of τ_v lost to the same-session feed interruption each time v forwards. We write $\mathbf{q}, \boldsymbol{\tau}, \boldsymbol{\lambda} \in \mathbb{R}^n$ for the vector forms.

Initially, each user v becomes a seed spontaneously with probability q_v and starts to propagate the content via the IC model. When a user shares the feed to others, its session value is reduced by $\lambda_v \tau_v$. That is, the net engagement is $(1 - \lambda_v) \tau_v$. The sum of net engagement over all activated nodes is the total engagement for the platform.

Intervention. The platform’s decision is a per-user forwarding propensity $\boldsymbol{\xi} = (\xi_1, \dots, \xi_n) \in [\xi_{10}, 1]^n$, where ξ_u is the conditional

probability that an activated user u forwards, and $\xi_{10} \in (0, 1)$ is an engineering floor below which the platform cannot suppress sharing. A chosen $\boldsymbol{\xi}$ is implemented through interface interventions that adjust the visibility of the sharing entry, introduce confirmation or recipient-selection steps, impose rate limits or cooldowns.

Given $\boldsymbol{\xi}$, the propagation proceeds in three stages. In *seeding*, we draw $Z_u \sim \text{Bernoulli}(q_u)$ independently for each $u \in V$ and let $S = \{u : Z_u = 1\}$ be the initially activated seed set. In the *cascade*, activation spreads by the IC model on G with edge probabilities $\{p_{u,v}\}$, modulated by $\boldsymbol{\xi}$. Each activated user u forwards independently with probability ξ_u , and if it forwards, each outgoing edge (u, v) activates v independently with probability $p_{u,v}$. The process *terminates* when no new activation occurs, and we write $A(\boldsymbol{\xi}) \subseteq V$ for the final activated set.

3.2 EMFC’s Objective

The platform’s objective is the expected total engagement earned under a control vector $\boldsymbol{\xi}$. Each user v contributes in two opposing ways. When activated, v generates session value τ_v from feed consumption, but each forward interrupts that session and costs a fraction λ_v of the value. Under the control configuration $\boldsymbol{\xi}$, an activated user v forwards with probability ξ_v .

For a single realization, the benefit from v is

$$B_v(\boldsymbol{\xi}) = \tau_v \cdot \mathbf{1}[v \in A(\boldsymbol{\xi})] \cdot (1 - \lambda_v Y_v),$$

where $Y_v \sim \text{Bernoulli}(\xi_v)$ indicates forwarding, conditionally independent of $\mathbf{1}[v \in A(\boldsymbol{\xi})]$ given the cascade history. Taking expectations and using $\mathbb{E}[\mathbf{1}[v \in A(\boldsymbol{\xi})] \cdot (1 - \lambda_v Y_v)] = \Pr[v \in A(\boldsymbol{\xi})] \cdot (1 - \lambda_v \xi_v)$ gives

$$J(\boldsymbol{\xi}) = \sum_{v \in V} \tau_v \cdot \Pr[v \in A(\boldsymbol{\xi})] \cdot (1 - \lambda_v \xi_v). \quad (1)$$

The factor $(1 - \lambda_v \xi_v)$ captures the self-incurred friction. An activated user v generates its full session value τ_v , but each forward reduces that value by the fraction λ_v . The two random sources behind $\Pr[v \in A(\boldsymbol{\xi})]$, edge transmission and seed sampling, are exogenous, while $\boldsymbol{\xi}$ governs both whether the cascade reaches v and how much v contributes once reached. We now state the optimization problem formally.

DEFINITION 3.1 (ENGAGEMENT MAXIMIZATION VIA FORWARD CONTROL). Given a directed social graph $G = (V, E)$, edge transmission probabilities $\{p_{u,v}\}_{(u,v) \in E}$, seed probabilities $\mathbf{q}$, session values $\boldsymbol{\tau}$, friction coefficients $\boldsymbol{\lambda}$, and a lower forwarding propensity ξ_{10} , the *Engagement Maximization via Forward Control (EMFC) problem* seeks a control vector $\boldsymbol{\xi}^* \in [\xi_{10}, 1]^n$ that maximizes the expected total engagement defined in (1). Formally,

$$\boldsymbol{\xi}^* \in \arg \max_{\boldsymbol{\xi} \in [\xi_{10}, 1]^n} J(\boldsymbol{\xi}). \quad (2)$$

3.3 Properties

We first establish two structural properties of EMFC that guide the algorithm design.

THEOREM 3.2 (MULTILINEARITY). For every $u \in V$ and any fixed $\boldsymbol{\xi}_{-u}$, there exist functions a_u and b_u , both independent of ξ_u , such that

$$J(\boldsymbol{\xi}) = a_u(\boldsymbol{\xi}_{-u}) + b_u(\boldsymbol{\xi}_{-u}) \xi_u. \quad (3)$$

Consequently, J is affine in each coordinate and is therefore multi-affine over $[\xi_{\text{lo}}, 1]^n$.

PROOF. Fix a user $u \in V$ and a control vector ξ_{-u} . Let $Y_u \sim \text{Bernoulli}(\xi_u)$ indicate whether u forwards after becoming activated. Let Ω_{-u} collect all randomness other than Y_u , including seed selection, live-edge realizations, and the forwarding decisions of all users other than u . For fixed ξ_{-u} , the distribution of Ω_{-u} does not depend on ξ_u .

For $y \in \{0, 1\}$ and a realization ω of Ω_{-u} , let $Z_u(y, \omega)$ denote the total engagement generated when $Y_u = y$ and $\Omega_{-u} = \omega$. Conditioning on Y_u gives

$$\begin{aligned} J(\xi_u, \xi_{-u}) &= (1 - \xi_u) \mathbb{E}_{\Omega_{-u}} [Z_u(0, \Omega_{-u})] + \xi_u \mathbb{E}_{\Omega_{-u}} [Z_u(1, \Omega_{-u})] \\ &= a_u(\xi_{-u}) + b_u(\xi_{-u}) \xi_u, \end{aligned} \quad (4)$$

where

$$a_u(\xi_{-u}) = \mathbb{E}_{\Omega_{-u}} [Z_u(0, \Omega_{-u})]$$

and

$$b_u(\xi_{-u}) = \mathbb{E}_{\Omega_{-u}} [Z_u(1, \Omega_{-u}) - Z_u(0, \Omega_{-u})].$$

Neither coefficient depends on ξ_u . Since this argument holds for every $u \in V$, the objective is multi-affine over $[\xi_{\text{lo}}, 1]^n$. $\square$

This coordinate-wise multilinearity imposes a strong structure on the optimal control. In particular, it leads to the following bang-bang optimality result, which reduces the continuous control space to a discrete set of endpoint decisions.

THEOREM 3.3 (BANG-BANG OPTIMALITY). *EMFC admits an optimal solution on the discrete vertex set $\xi^\circ \in \{\xi_{\text{lo}}, 1\}^n$.*

PROOF. Since J is continuous and $[\xi_{\text{lo}}, 1]^n$ is compact, EMFC admits a global maximizer ξ° . Fix any coordinate $u \in V$ and consider varying only ξ_u while all other coordinates remain at their values in ξ° . By Theorem 3.2, J is affine in ξ_u on $[\xi_{\text{lo}}, 1]$. Its maximum over this interval is therefore attained at either ξ_{lo} or 1. We may replace ξ_u° by an endpoint without decreasing the objective value. Applying this argument successively to every coordinate produces a vector $\tilde{\xi} \in \{\xi_{\text{lo}}, 1\}^n$ such that $J(\tilde{\xi}) \geq J(\xi^\circ)$. Since ξ° is a global maximizer, equality must hold. Therefore, $\tilde{\xi}$ is also a global maximizer of EMFC. $\square$

Theorem 3.3 reduces the continuous control problem to a binary decision for each user. The platform either encourages forwarding by setting $\xi_u = 1$ or suppresses it by setting $\xi_u = \xi_{\text{lo}}$.

This reduction collapses the search space to 2^n vertices, but EMFC remains intractable. The objective is not submodular, and the following theorem rules out any constant-factor approximation unless $P = NP$.

THEOREM 3.4 (INAPPROXIMABILITY). *Unless $P = NP$, no polynomial-time algorithm can guarantee a constant-factor multiplicative approximation for EMFC in general.*

PROOF. Take $\tau_v \equiv 1$ and $\lambda_v \equiv 1$ for each $v \in V$, so $J(\xi) = \sum_v \Pr[v \in A(\xi)](1 - \xi_v)$. Restricting to the binary domain $\xi \in \{0, 1\}^n$, the objective becomes the difference of the influence spread function, which is submodular under the IC model, and a modular penalty on the forwarding set. Maximizing such a difference is known to admit no constant-factor approximation in general [14], and this hardness carries over to EMFC. $\square$

Although Theorem 3.4 rules out a constant-factor approximation to the global optimum, it does not preclude finding a solution that is stable under individual control changes. Since Theorem 3.3 restricts the search to the vertices of the feasible box, we consider a coordinate-local optimum at which switching any single user's control cannot improve the objective.

To state this formally, we introduce the flip notation. For a vertex $\xi \in \{\xi_{\text{lo}}, 1\}^n$, let $\xi^{\oplus u}$ denote the vector obtained by switching ξ_u to the other endpoint while leaving all other coordinates unchanged.

DEFINITION 3.5 (COORDINATE-LOCAL OPTIMUM). *A control vector $\xi \in \{\xi_{\text{lo}}, 1\}^n$ is a coordinate-local optimum if*

$$J(\xi^{\oplus u}) \leq J(\xi) \quad \text{for every } u \in V. \quad (5)$$

The coordinate-wise affinity of J gives an equivalent characterization in terms of the sign of its partial derivative.

LEMMA 3.6 (CHARACTERIZATION OF COORDINATE-LOCAL OPTIMA). *A point $\xi^\circ \in \{\xi_{\text{lo}}, 1\}^n$ is a coordinate-local optimum if and only if, for every $u \in V$,*

$$\frac{\partial J}{\partial \xi_u}(\xi^\circ) \geq 0 \quad \text{if } \xi_u^\circ = 1, \quad \frac{\partial J}{\partial \xi_u}(\xi^\circ) \leq 0 \quad \text{if } \xi_u^\circ = \xi_{\text{lo}}.$$

If the derivative is zero, both endpoints are optimal for u .

PROOF. Fix $u \in V$ and a point $\xi \in \{\xi_{\text{lo}}, 1\}^n$. By Theorem 3.2, J is affine in ξ_u when all other coordinates are fixed. The change in objective from flipping ξ_u to the opposite endpoint is

$$J(\xi^{\oplus u}) - J(\xi) = (1 - \xi_u) \frac{\partial J}{\partial \xi_u}(\xi).$$

If $\xi_u = 1$, flipping to ξ_{lo} cannot improve the objective exactly when the partial derivative is nonnegative. If $\xi_u = \xi_{\text{lo}}$, flipping to 1 cannot improve the objective exactly when the partial derivative is non-positive. Applying this to every coordinate proves the result. $\square$

Lemma 3.6 is the operational basis of the algorithms in Section 4, which reduce the search to repeatedly estimating the sign of the $\partial J / \partial \xi_u$ at the current iterate.

4 METHODS

In this section, we develop algorithms for EMFC. We begin by introducing a coordinate ascent framework, which repeatedly compares the two endpoint choices for each user and retains the better one. Building on aforementioned properties, we prove that it converges to a coordinate-local optimum. However, evaluating the objective is $\#P$ -hard, so each endpoint comparison must be estimated from samples. Drawing fresh Monte Carlo samples or RR-sets for every comparison is expensive and does not define a fixed empirical objective. As a result, accepted switches need not produce a monotone sequence, and the usual convergence argument for coordinate ascent no longer applies. We discuss this issue in § 4.1.

We first address the problem by fixing a pool of live-edge realizations and reusing it throughout the search. This produces an effective baseline solver (§ 4.2), but evaluating a switch still requires costly forward propagation on the sampled graphs. Reverse-reachable sampling is more efficient, but a standard RR set discards the connectivity information needed to evaluate different control vectors.

Algorithm 1 Coordinate ascent framework

Input: Objective function J , user set V , and lower endpoint ξ_{lo}

Output: A control vector $\xi^* \in \{\xi_{lo}, 1\}^n$

```
 $\xi \leftarrow (1, \dots, 1)$ 
repeat
  improved  $\leftarrow$  false
  for each  $u \in V$  do
     $\Delta_u \leftarrow J(\xi^{\oplus u}) - J(\xi)$ 
    if  $\Delta_u > 0$  then
       $\xi \leftarrow \xi^{\oplus u}$ 
  improved  $\leftarrow$  true
until not improved
return  $\xi$ 
```

To obtain an efficient unbiased estimator for the objective function, we introduce the *gated reverse-reachable graph* (GRRG) in § 4.3. A fixed pool of these samples can evaluate all control vectors while preserving the connectivity needed for each endpoint comparison. We show that CA-GRRG returns an approximate coordinate-local optimum. Finally, repeatedly scanning every node during the coordinate ascent process remains costly on large graphs. We mitigate this cost in § 4.4 by decomposing the marginal gain and pruning users whose preferred endpoint is already determined irrespective of other coordinates.

4.1 Coordinate Ascent

Theorem 3.3 allows us to reduce the search to the vertex set $\{\xi_{lo}, 1\}^n$, and Lemma 3.6 states that the preferred endpoint for each user is determined by the sign of $\partial J / \partial \xi_u$. This naturally leads to coordinate ascent, which starts from an initial configuration, sweeps over users, and flips ξ_u to the opposite endpoint whenever the objective increases. The search stops when no single-coordinate flip can improve the objective.

The pseudocode of coordinate ascent is given by Algorithm 1. The algorithm starts from an initial configuration and repeatedly sweeps the users. For each user u , it flips ξ_u to the opposite endpoint whenever the objective increases. The search terminates when no single coordinate flip yields an improvement, and Lemma 3.6 then certifies a coordinate-local optimum.

THEOREM 4.1 (COORDINATE-LOCAL OPTIMUM). *Given exact oracle access to J , Algorithm 1 terminates after a finite number of sweeps and returns a point $\xi^* \in \{\xi_{lo}, 1\}^n$ that is a coordinate-local optimum of J over the vertex set.*

However, computing the objective is #P-hard under the independent cascade model [2], since it requires the activation probabilities $\Pr[v \in A(\xi)]$, and the marginal α_u inherits this hardness. Every flip decision must therefore rely on an estimate. A naive way to estimate a flip is to draw a batch of Monte Carlo simulations, or a batch of reverse-reachable sets, afresh for each user at each sweep, compare $\widehat{J}(\xi_u = 1)$ against $\widehat{J}(\xi_u = \xi_{lo})$, and keep the better endpoint. This scheme is both expensive and, more seriously, unreliable, for two coupled reasons. First, the decision depends only on the sign of $\Delta_u = J(\xi_u = 1) - J(\xi_u = \xi_{lo})$, but if the two sides are estimated from independent samples their errors do not cancel and the variance of the difference is the *sum* of the two variances. For the many users

Table 2: Failure of drawing fresh samples at each comparison.

		200	400	600	800	1000
Improvement	Naive MC	13.8	17.8	44.5	47.8	56
	Fixed snapshots	696.8	699.4	698.8	701.3	701.1
Time (s)	Naive MC	2,236	4,468	6,722	8,971	11,230
	Fixed snapshots	240	468	721	937	1,080
Sweeps	Naive MC	5	5	5	5	5
	Fixed snapshots	2.9	2.8	2.9	2.8	2.5

whose true marginal is small, this doubled noise routinely swamps the sign, so the flip decision is often not accurate. Second, with fresh randomness at every step there is no single function being optimized, since the quantity a flip is accepted against changes from one step to the next, so accepted flips need not improve any fixed objective and the sweep can oscillate indefinitely rather than settle. Errors do not average out over sweeps but accumulate.

We run a small illustrative experiment on the GRQC dataset to expose the failure mode of this naive scheme. In this experiment, the maximum number of sweeps is set to 5. We vary the number of simulations / snapshots from 200 to 1000. The results are shown in Table 2, which shows that re-drawing fresh samples at every sweep causes the algorithm to exhaust all five scans with only marginal improvement and increasing the simulation budget per sweep merely inflates the running time without yielding commensurate gains.

A simple remedy resolves both issues. Fixing a pool of samples once and reusing it for every comparison yields a deterministic estimator $\widehat{J}$ that the algorithm optimizes monotonically. Every accepted flip increases $\widehat{J}$, and the search is guaranteed to terminate.

4.2 A Baseline: CA-LEG

The most direct frozen structure fixes a pool of live-edge graphs. For each sample it draws a live-edge graph h_r from the IC model and fixes a seed coin and a forwarding coin for every node, so that flipping one control changes only that user’s forwarding and not the shared randomness. A candidate flip of u is evaluated by an incremental BFS on each h_r that adds or removes u ’s outgoing edges, and the change in J is read off exactly on the pool.

This baseline method, named coordinate ascent with live-edge graphs (CA-LEG), proceeds as follows. We freeze a pool of live-edge graphs (e.g., 1,000 snapshots). Each graph carries a fixed seed set. Given a control configuration, activation propagates along retained edges, and an activated node continues spreading only if its retention gate is at most the control value of the corresponding user. The sample objective averages, over all frozen graphs, the sum of per-node benefits collected by activated nodes. Starting from the all-ones configuration, for each user u in each sweep, the algorithm temporarily flips ξ_u between 1 and ξ_{lo} , re-evaluates the objective on the same pool to obtain the average change $\Delta \widehat{J}$, and accepts the flip if $\Delta \widehat{J} > 0$. Accepted flips update the active sets incrementally, and otherwise the original value is restored. Sweeps continue until a full pass yields no improving flip or a maximum sweep limit is reached. The final ξ are returned.

This solver is effective, and it preserves the full structure of every realization and, with enough samples, reaches a coordinate-local optimum of the quality we target. However, this process incurs high

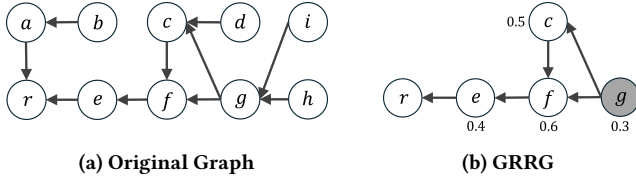


Figure 1: An example of GRRG sampling.

computational cost. Each flip test maintains the entire activated set and its incremental BFS traverses the downstream reachable nodes of u , so a single sweep touches large portions of the graph on every sample, which makes it hard to scale to large networks.

The standard remedy for this cost in influence maximization is reverse influence sampling (RIS). A plain RR set, however, records only the *set* of nodes that can reach a sampled root under a fixed activation. It collapses the internal path structure and is tied to one activation pattern, so it cannot answer the question our marginal poses, namely whether the root is still reached once a particular user’s forwarding is gated at a given control ξ_u .

4.3 CA with Gated Reverse-Reachable Graphs

Incorporating structural information into reverse influence sampling is not new in itself [20]. Adapting this idea to our setting, however, requires solving three problems that do not arise in standard IM. First, the ξ -gate randomness must be frozen at sampling time so that the same pool can be evaluated at different control vectors. Second, not every node that is reverse-reachable from the root needs to be stored. Only nodes that lie on a forward path from some q -activated seed to the root affect the engagement estimate, so the reverse BFS can drop dead-end branches whose reverse-ancestry contains no seed. Third, the standard benefit-aware sampling [22] samples each root node with probability proportional to its benefit. However, in our case, the benefit of each node is related to the control vector, which means that we may need to re-sample each time ξ is updated, bringing extra overhead.

To address these issues, we propose a concept named *gated reverse reachable graph* (GRRG), which keeps the reverse, value-weighted sampling that makes RR estimation efficient.

GRRG sampling. A GRRG is a tuple

$$R = (r, H, V_R, E_R, \xi^g),$$

where r is the root, H is the set of seeds encountered during reverse traversal, (V_R, E_R) is the retained live-edge subgraph, and ξ^g records the gate threshold of each non-root node.

Algorithm 2 gives the pseudocode for generating a random GRRG sample. Let $T = \sum_{u \in V} \tau_u$. The root r is drawn by selecting each node v with probability τ_v/T . Starting from r , we perform a reverse BFS. When an in-neighbor u of the current node v is examined, the edge (u, v) is retained independently with probability $p_{u,v}$. Note that the reverse BFS is independent of ξ , which is equivalent to sampling under the all-ones configuration $\xi = 1$.

When a node u is first reached, it is marked as a q -hit with probability q_u . If so, u is added to H and the reverse traversal stops along this branch. Otherwise, the search continues through its

sampled live in-edges. Thus, H contains the seeds encountered on the reverse paths to r .

The reverse traversal may discover branches without any seed node, which therefore cannot lead to root activation. We prune these branches by performing a forward BFS from H on the sampled edges. The nodes and edges reached by this traversal form (V_R, E_R) , i.e., the union of the sampled live paths from H to r . If H is empty, the algorithm returns an empty sample.

Finally, each node $u \in V_R \setminus \{r\}$ draws an independent gate threshold $\xi^g(u) \sim \text{Uniform}[0, 1]$. This map is fixed once the sample is built. Under a control vector ξ , node u forwards when $\xi_R^g(u) \leq \xi_u$. The root does not need a gate because its own forwarding does not affect whether it is activated. The sample R is active under ξ if some node in H can reach r through a path on which every non-root node has an open gate.

Evaluating $J(\xi)$ reduces to a deterministic reachability question on this fixed subgraph. Each sample contributes the term $1 - \lambda_{R,r} \xi_{R,r}$ whenever an active seed reaches the root through open gates. Because the live-edge structure and the gate thresholds are frozen, changing ξ only alters which nodes are active without modifying the underlying graph. A single pool therefore provides an unbiased estimate of J at any control vector, not merely at $\xi = 1$.

With GRRGs, we can formalize the following estimator.

LEMMA 4.2 (UNBIASED ESTIMATOR). *Let $T = \sum_{v \in V} \tau_v$, and let R be a random gated reverse-reachable graph whose root $R.r$ is drawn with $\Pr[R.r = v] = \tau_v/T$. Let $I_R(\xi)$ indicate whether some q -hit seed reaches $R.r$ in R under the control vector ξ . Then, for every fixed $\xi \in [\xi_{10}, 1]^n$,*

$$J(\xi) = T \mathbb{E} \left[(1 - \lambda_{R,r} \xi_{R,r}) I_R(\xi) \right]. \quad (6)$$

Example 4.3. Figure 1 shows a GRRG rooted at node r . Suppose every edge has propagation probability 1, and the only seed is g , so $q_g = 1$ while all other q -values are 0. We sample root r with probability $\frac{\tau_r}{\sum_{v \in V} \tau_v}$ and run a reverse traversal. All visited nodes are reverse-reachable, but the expansion stops at g because g is already active as a seed. Consequently, nodes i and h are discarded. We then run a forward traversal from g and keep only the nodes and edges that lie on a forward path from the seed to the root, which discards nodes a , b , and d . Finally, every non-root node draws a uniform ξ -gate, shown next to the corresponding node. The root needs no ξ -gate because it does not forward to any other node in this sample. Given this sample, when the forwarding propensity is $\xi = 0.5 \cdot \mathbf{1}$, node f is blocked because its gate $0.6 > 0.5$, so the contribution of this sample is 0. When the propensity is $\xi = 0.7 \cdot \mathbf{1}$, every node forwards and g reaches the root successfully, and the weight of this GRRG is then $1 - \lambda_r \xi_r$.

With Theorem 4.2, after sampling a large number of GRRGs, we can estimate J by

$$\Lambda_{\mathcal{R}}(\xi) \triangleq \sum_{R \in \mathcal{R}} (1 - \lambda_{R,r} \xi_{R,r}) I_R(\xi). \quad (7)$$

Running CA on $\Lambda_{\mathcal{R}}(\cdot)$ will give an approximate coordinate-local optimum. In the following, we show our implementation of CA-GRRG and analyze the theoretical guarantees.

CA-GRRG. Coordinate ascent with the GRRG (CA-GRRG) estimator works as follows. The algorithm initializes every user to the

Algorithm 2 Sample a random GRRG

Input: Graph $G = (V, E)$, live-edge probs. $p_{u,v}$, seed probs. q , root weights τ

Output: GRRG $R = (r, H, V, E, \xi_R^g)$ at $\xi \equiv 1$;

- 1: Sample root $r \in V$ with $\Pr[r = u] = \tau_u/T$, where $T = \sum_{w \in V} \tau_w$
- 2: $H \leftarrow \emptyset$; $V \leftarrow \{r\}$; $E \leftarrow \emptyset$; enqueue r into Q
- 3: **while** Q not empty **do**
- 4: pop u from Q
- 5: **if** $u < |q|$ **and** $\text{RAND}() < q_u$ **then**
- 6: $H \leftarrow H \cup \{u\}$ *$\triangleright$ q-hit: record seed, do not expand u*
- 7: **else**
- 8: **for** each in-neighbor v with $\text{RAND}() < p_{v,u}$ **do**
- 9: $E \leftarrow E \cup \{(v, u)\}$; $V \leftarrow V \cup \{v, u\}$; enqueue unseen v into Q
- 10: **if** $H = \emptyset$ **then**
- 11: **return** empty R
- 12: Forward BFS from H on E ; only nodes and edges visited in this traversal are kept as the final V and E
- 13: **for** $u \in V \setminus \{r\}$ **do**
- 14: $\xi_R^g(u) \leftarrow \text{UNIF}(0, 1)$
- 15: **return** R

Algorithm 3 CA-GRRG

Input: Number of samples θ ; λ ; tolerance $\xi_{\text{lo}} \in (0, 1]$; active nodes $U \subseteq V$ (default $U = V$)

Output: Forward propensity vector ξ

- 1: $\xi \leftarrow 1$
- 2: Sample θ GRRGs by Algorithm 2.
- 3: For each $R \in \mathcal{R}$: $w(R, \xi) \leftarrow (1 - \lambda_{R,r} \xi_{R,r})$
- 4: $\Psi \leftarrow \sum_{R \in \mathcal{R}} w(R, \xi)$
- 5: **repeat**
- 6: improved $\leftarrow \text{FALSE}$
- 7: Shuffle U into random order π
- 8: **for** each $u \in \pi$ **do**
- 9: $\Psi_1 \leftarrow \Psi - \sum_{R \in \mathcal{R}_u} w(R, \xi) + \sum_{R \in \mathcal{R}_u} w(R, \xi_{u \leftarrow 1})$
- 10: $\Psi_{\text{lo}} \leftarrow \Psi - \sum_{R \in \mathcal{R}_u} w(R, \xi) + \sum_{R \in \mathcal{R}_u} w(R, \xi_{u \leftarrow \xi_{\text{lo}}})$
- 11: $\xi_u^* \leftarrow \arg \max_{\xi' \in \{1, \xi_{\text{lo}}\}} \{\Psi_1, \Psi_{\text{lo}}\}$
- 12: **if** $\xi_u^* \neq \xi_u$ **then**
- 13: $\xi_u \leftarrow \xi_u^*$; update $w(R, \xi)$ for $R \in \mathcal{R}_u$
- 14: $\Psi \leftarrow \Psi_1$ if $\xi_u^* = 1$ else Ψ_{lo}
- 15: improved $\leftarrow \text{TRUE}$
- 16: **until** not improved
- 17: **return** ξ

encourage endpoint $\xi \equiv 1$ and builds the frozen GRRG sample pool once. The initial estimate $\Lambda_{\mathcal{R}}(\xi)$ is the sum of the per-sample weights $w(R, \xi) = (1 - \lambda_{R,r} \xi_{R,r})$ over all samples R that are reached by at least one active seed. In each sweep, users are visited in random order, which is a standard technique [28]. For each user u , the algorithm evaluates the two candidate endpoints by recomputing the estimate incrementally, subtracting the current contribution of the samples that involve u and adding back their contribution under the flipped endpoint. The endpoint that yields the higher estimate is retained. If a flip occurs, the per-sample weights are updated and the sweep continues. The procedure repeats until a full sweep

makes no change, at which point the current ξ is a coordinate-local optimum of $\Lambda_{\mathcal{R}}(\cdot)$. The pseudocode is presented by Algorithm 3, where we use Ψ to denote the dynamically maintained $\Lambda_{\mathcal{R}}(\xi)$.

Theoretical guarantees. We now analyze the quality of the solution returned by the GRRG-based coordinate ascent. Theorem 3.4 rules out a worst-case constant-factor multiplicative approximation, so we settle for a relaxed notion of optimality that is both attainable and meaningful in this setting. A point is an ε -additive coordinate-local optimum if no single-coordinate flip improves the objective by more than ε , defined as follows.

DEFINITION 4.4 (ε -ADDITIVE APPROXIMATE COORDINATE-LOCAL OPTIMUM). Let $\mathcal{X} = \{\xi_{\text{lo}}, 1\}^n$. For any $\xi \in \mathcal{X}$, define its single-coordinate neighborhood as $\mathcal{N}_1(\xi) = \{\xi' \in \mathcal{X} : \|\xi' - \xi\|_0 = 1\}$. A vector $\xi \in \mathcal{X}$ is an ε -additive approximate coordinate-local optimum if

$$J(\xi') \leq J(\xi) + \varepsilon \quad \text{for every } \xi' \in \mathcal{N}_1(\xi). \quad (8)$$

We then present a lemma that bounds the estimation error, laying the foundation for the final approximation error analysis.

LEMMA 4.5. Let $T = \sum_{v \in V} \tau_v$ and let $\mathcal{R}$ be a pool of independent GRRG samples. If

$$|\mathcal{R}| \geq \frac{(8 + 2\varepsilon) T (\ln(1/\delta) + n \ln 2 + \ln 2)}{\varepsilon^2 n},$$

then, with probability at least $1 - \delta$,

$$\left| \frac{T}{|\mathcal{R}|} \Lambda_{\mathcal{R}}(\xi) - J(\xi) \right| \leq \frac{\varepsilon n}{2}$$

holds for any control vector ξ .

With $\Lambda_{\mathcal{R}}(\cdot)$ being an unbiased estimator for $J(\cdot)$, the coordinate-local optimality of the output of Algorithm 3 on $\Lambda_{\mathcal{R}}$ transfers to an additive guarantee on J .

THEOREM 4.6 (APPROXIMATE COORDINATE-LOCAL OPTIMALITY). Suppose the sample size satisfies the condition in Lemma 4.5. Then Algorithm 3 terminates after finitely many flips and, with probability at least $1 - \delta$, returns a control vector $\hat{\xi}$ that is an εn -additive coordinate-local optimum of J .

PROOF. Let $\hat{J}(\xi) = \frac{T}{|\mathcal{R}|} \Lambda_{\mathcal{R}}(\xi)$. We first argue termination. Every accepted flip strictly increases $\hat{J}$, and $\hat{J}$ takes finitely many values on the vertex set $\{\xi_{\text{lo}}, 1\}^n$, so no vertex is visited twice and the algorithm stops after finitely many flips.

Condition on the event of Lemma 4.5, which occurs with probability at least $1 - \delta$ and gives

$$|\hat{J}(\xi) - J(\xi)| \leq \frac{\varepsilon n}{2} \quad \text{for all } \xi \in \{\xi_{\text{lo}}, 1\}^n.$$

Let $\hat{\xi}$ be the returned vector and let ξ' be any feasible vector differing from $\hat{\xi}$ in exactly one coordinate. The algorithm stops only when no single flip is accepted, so $\hat{J}(\xi') \leq \hat{J}(\hat{\xi})$. Applying the accuracy bound to ξ' and to $\hat{\xi}$ in turn,

$$J(\xi') \leq \hat{J}(\xi') + \frac{\varepsilon n}{2} \leq \hat{J}(\hat{\xi}) + \frac{\varepsilon n}{2} \leq J(\hat{\xi}) + \varepsilon n.$$

Since ξ' was arbitrary, $J(\xi') - J(\hat{\xi}) \leq \varepsilon n$ holds for every single-coordinate flip, which is exactly the definition of an εn -additive coordinate-local optimum. $\square$

Algorithm 4 Pruning stage of CA-GRRG⁺

Input: GRRG collection $\mathcal{R}$ with CRN gates ξ_R^g ; $\{\lambda_u\}_{u \in V}$; tolerance $\xi_{lo} \in (0, 1]$

Output: For each u : fix $\xi_u = 1$, fix $\xi_u = \xi_{lo}$, or leave uncertain

```

1:  $K_{lb}[u], K_{ub}[u], n_1[u], n_{lo}[u] \leftarrow 0$  for all  $u \in V$ 
2: for non-empty  $R \in \mathcal{R}$  do
3:    $v \leftarrow R.root$ 
4:   if CONNECTED( $R, \xi \equiv 1$ ) then
5:      $n_1[v] \leftarrow n_1[v] + 1$ 
6:   if CONNECTED( $R, \xi \equiv \xi_{lo}$ ) then
7:      $n_{lo}[v] \leftarrow n_{lo}[v] + 1$ 
8:    $B \leftarrow \text{ARTICULATIONBLOCKS}(R, \xi \equiv 1) \triangleright \text{Tarjan blocks under } \xi^g \text{ gates at } \xi = 1$ 
9:   for each  $u \in R.V \setminus \{v\}$  do
10:     $K_{ub}[u] \leftarrow K_{ub}[u] + (1 - \lambda_v \xi_{lo})$ 
11:    if  $B(u)$  and QHIT( $R, \xi_u=1, \xi_{-u}=\xi_{lo}$ ) then
12:       $K_{lb}[u] \leftarrow K_{lb}[u] + (1 - \lambda_v)$ 
13: for each  $u \in V$  do
14:    $L_{ub}[u] \leftarrow \lambda_u \cdot n_1[u]; L_{lb}[u] \leftarrow \lambda_u \cdot n_{lo}[u]$ 
15:    $\underline{\alpha}[u] \leftarrow K_{lb}[u] - L_{ub}[u] \triangleright \text{gradient lower bound at } \xi_u = 1$ 
16:    $\bar{\alpha}[u] \leftarrow K_{ub}[u] - L_{lb}[u] \triangleright \text{gradient upper bound at } \xi_u = \xi_{lo}$ 
17:   if  $\underline{\alpha}[u] > 0$  then
18:     fix  $\xi_u \leftarrow 1$ 
19:   else if  $\bar{\alpha}[u] < 0$  then
20:     fix  $\xi_u \leftarrow \xi_{lo}$ 
21:   else
22:     mark  $u$  as active
23: return  $\xi$ 

```

Time complexity. Let γ denote the average per-sample cost of generating one RR-set. The sampling stage of CA-GRRG takes $O(\theta \cdot \gamma)$ time. Coordinate ascent runs for s sweeps over n active nodes. In the worst case, evaluating a flip for node u re-inspects every sample that contains u , each costing $O(\gamma)$ for a reachability check on the GRRG. Summing across all nodes in one sweep gives a naive bound of $O(\theta \cdot \gamma^2)$. The overall worst-case runtime is therefore $O(s \theta \gamma^2)$. By Lemma 4.5, $\theta = \Theta(Tn \log(1/\delta)/(\epsilon^2 C))$, yielding $O(\frac{s T \log(1/\delta)}{\epsilon^2} \gamma^2)$, to obtain an ϵn -additive approximate coordinate-local optimum. This complexity is pessimistic since the samples are typically small and nodes appear only in samples where they lie on a seed-to-root path. In our experiments, the algorithms finish within minutes for million-size graphs.

4.4 Pruning via Gradient Decomposition

Even with a reusable estimator, sweeping over every user in each round dominates the cost on large graphs, and much of that work falls on users whose endpoint is already settled. In this subsection, we aim to lock these users before the sweep begins. Because coordinate ascent optimizes the estimator $\Lambda_{\mathcal{R}}(\cdot)$ defined by the frozen pool, the locks are established directly on $\Lambda_{\mathcal{R}}(\cdot)$ and do not involve any additional statistical error.

The coordinate-wise partial derivative of $\Lambda_{\mathcal{R}}(\xi)$ could be decomposed into two contributions:

$$\frac{\partial \Lambda_{\mathcal{R}}(\xi)}{\partial \xi_u} = \underbrace{\sum_{\substack{R \in \mathcal{R} \\ R.r \neq u}} (1 - \lambda_{R,r} \xi_{R,r}) \frac{\partial I_R(\xi)}{\partial \xi_u}}_{K_u(\xi) : \text{downstream gain}} - \underbrace{\lambda_u \sum_{\substack{R \in \mathcal{R} \\ R.r = u}} I_R(\xi)}_{L_u(\xi) : \text{self-friction}}. \quad (9)$$

The first term captures downstream activation changes through samples where u is an intermediate node; the second term captures the direct benefit loss on samples rooted at u .

For notation convenience, we write $\frac{\partial I_R(\xi)}{\partial \xi_u}$ as $\nabla_u I_R(\xi)$, and we have

$$\nabla_u I_R(\xi) \triangleq \frac{\partial I_R(\xi)}{\partial \xi_u} = I_R(\xi^{u \leftarrow 1}) - I_R(\xi^{u \leftarrow 0}), \quad (10)$$

where $\xi^{u \leftarrow x}$ is the vector obtained by setting $\xi_u = x$ and leaving all other coordinates unchanged. Recall that $I_R(\xi)$ indicate whether some seed reaches $R.r$ in R under the control vector ξ .

A sample contributes to K_u only when it lies outside $\mathcal{R}_u$, since the gate of u cannot influence whether u itself was reached, and it contributes to L_u only when it lies inside.

The endpoint at u is decided by the sign of $K_u - L_u$. If that sign is constant over the whole feasible box, the decision is settled before the sweep starts. Write $\ell = \xi_{lo} \mathbf{1}$ for the lowest admissible control. In the following, we propose the bounds for K_u and L_u , respectively.

Bounding the downstream gain. For the upper bound of K_u , we fall back on two facts that hold everywhere. Opening the gate of u can only matter in a sample containing u as a non-root node, so $\nabla_u I_R(\xi) \leq \mathbf{1}[u \in V_R \setminus \{R.r\}]$, and the root value satisfies $1 - \lambda_{R,r} \xi_{R,r} \leq 1 - \lambda_{R,r} \xi_{lo}$ because $\xi_{R,r} \geq \xi_{lo}$. Crediting every reached sample that contains u at its largest possible root value gives

$$\bar{K}_u = \sum_{R \in \mathcal{R}_u, R.r \neq u} (1 - \lambda_{R,r} \xi_{lo}). \quad (11)$$

For the lower bound, a sample may be credited to K_u at every admissible control only if two conditions hold together. First, no path from any seed to the root bypasses u . A Tarjan-based block decomposition run on each reached sample at $\xi \equiv \mathbf{1}$ marks the nodes whose removal disconnects every q -hit seed from the root, so a marked node satisfies $I_R(\mathbf{1}^{u \leftarrow 0}) = 0$, and since lowering controls cannot create new paths, no bypass appears at any weaker configuration. Second, a path through u already exists at the bottom of the box (i.e., $\xi = \ell$). We set $\xi_u = 1$, fix every other coordinate at ξ_{lo} , and test whether the sample is still reached, which is precisely $I_R(\ell^{u \leftarrow 1}) = 1$. Raising any other coordinate only helps, so the path survives at every admissible control. Crediting only the samples that pass both tests, each at the smallest possible root value $1 - \lambda_{R,r}$, gives

$$\underline{K}_u = \sum_{R \in \mathcal{R}_u, R.r \neq u} (1 - \lambda_{R,r}) I_R(\ell^{u \leftarrow 1}) [1 - I_R(\mathbf{1}^{u \leftarrow 0})]. \quad (12)$$

Bounding the self-friction. This term is far simpler. L_u counts the samples rooted at u , scaled by λ_u , and that count is monotone in the forward propensity of other users. Opening a gate can only add routes into u , never remove one, so the count is maximal at $\xi \equiv \mathbf{1}$ and minimal at $\xi \equiv \ell$, which yields

$$\underline{L}_u = \lambda_u \sum_{R \in \mathcal{R}_u, R.r = u} I_R(\ell), \quad \bar{L}_u = \lambda_u \sum_{R \in \mathcal{R}_u, R.r = u} I_R(\mathbf{1}). \quad (13)$$

Note that the gate of u itself plays no role here, since whether v was reached does not depend on whether v forwards afterwards. Both counts come directly from the reachability flags evaluated at the two extreme controls, so no extra traversal is needed.

After sampling a collection of GRRGs, we compute $\underline{K}_u, \bar{K}_u, \underline{L}_u$ and $\bar{L}_u$ for each node u . For any u , if $\underline{K}_u - \bar{L}_u > 0$, ξ_u can be locked to 1, and if $\bar{K}_u - \underline{L}_u < 0$, ξ_u can be safely fixed as ξ_{10} . The pseudocode of this pruning stage is given in Algorithm 4, where arrays K_{ub} , K_{lb} and L_{ub} , L_{lb} maintain the bounds on the downstream gain and self-friction terms. The returned configuration ξ and the set of active nodes are then passed to CA-GRRG, giving the accelerated variant CA-GRRG⁺.

5 ADDITIONAL RELATED WORK

Influence maximization. The idea of exploiting word-of-mouth effects in social networks to shape consumer behavior is first introduced by Domingos and Richardson [8]. Kempe et al. [16] later formalize the influence maximization (IM) problem, which aims to select k seed users to maximize expected influence spread under a given diffusion model. They prove that this problem is NP-hard in general and propose a greedy algorithm grounded in Monte Carlo simulation for the IC and LT model, which provides the now-classic $(1 - 1/e)$ approximation guarantee for monotone submodular objectives. A long line of subsequent work seeks faster practical solvers [2, 3, 5, 7, 9, 10, 15, 24]. Many of these methods rely on heuristics and therefore do not carry approximation guarantees. A major theoretical advance comes with reverse influence sampling (RIS), introduced by Borgs et al. [1], which reduces influence estimation to a maximum-coverage problem over sampled reverse reachable sets. This framework inspires a family of RIS-based methods that scale to very large graphs while retaining strong theoretical guarantees [11, 13, 23, 25–27].

Influence-aware continuous control. Beyond discrete seed selection, several studies optimize continuous interventions in influence propagation. Yang et al. [29] formulate continuous influence maximization as the allocation of user-level discounts, where each discount determines the probability that a user becomes an initial adopter. Chen et al. [6] incorporate the value of unspent marketing budget into this formulation and combine gradient optimization with reverse influence sampling. A related scalable extension discretizes the strategy space into a lattice and uses RIS to optimize probabilistic seed activation [4]. These studies use continuous decisions to control initial adoption. In contrast, EMFC controls each user’s forwarding propensity after activation, allowing the platform to intervene throughout the cascade. EMFC also captures the engagement loss borne by the forwarder, which is not considered in existing continuous influence-control models.

6 EXPERIMENTS

In this section, we conduct extensive experiments on 10 real-world datasets to evaluate the performance of the proposed algorithms. All algorithms are implemented in C++ with O3 optimization.

Table 3: Datasets ($K = 10^3$, $M = 10^6$, $B = 10^9$)

Name	n	m	Type	Avg. Degree
Facebook	4.0K	88.2K	Undirected	43.7
GRQC	5.2K	14.5K	Undirected	5.5
Wiki-Vote	7.1K	103.7K	Directed	29.2
HepTh	27.8K	352.8K	Directed	12.7
Epinions	75.9K	508.8K	Directed	13.4
EuAll	265.2K	420.0K	Directed	3.2
Stanford	281.9K	2.3M	Directed	16.4
DBLP	317.1K	1.0M	Undirected	6.6
YouTube	1.1M	3.0M	Undirected	5.3
Pokec	1.6M	30.6M	Directed	37.5

6.1 Experimental Setup

Environment. The experiments are conducted on a Linux machine with Intel Xeon(R) 8377C@3.0GHz and 512GB RAM.

Datasets. We conduct experiments on 10 public datasets [17, 19], with statistics summarized in Table 3.

Parameters. The propagation probability $p_{u,v}$ of each edge is set to the inverse of v ’s in-degree (the weighted-cascade model), a widely-adopted choice in the literature [2, 11, 13, 16, 25–27]. Unless otherwise specified, the seeding probability q_u of each node is drawn from a truncated exponential distribution on $[0, 1]$ with scale 1, the engagement weight is $\tau_u = \text{clip}_{[1,5]}((q_u + \zeta_u) \cdot 5)$ with $\zeta_u \sim \mathcal{N}(0, 1)$, and the forwarding cost is $\lambda_u \sim U[0, 0.5]$. This parameterization follows the intuition that users who are more likely to start a session (higher q_u) also tend to have higher engagement, though individual variation is introduced through the Gaussian noise ζ_u . The maximum number of sweeps is fixed at 10 for all CA-based algorithms. For CA-LEG, the number of snapshots is $r = 1,000$. The failure probability δ for CA-GRRG and CA-GRRG⁺ is set to $1/n$. $\xi_{10} = 0.5$ by default. The estimation-error parameter ϵ is 0.5 and the additive-error scale is ϵn . All reported engagement values are estimated from 10,000 independent Monte Carlo simulations and averaged over 10 random seeds.

Algorithms. In the experiments, we implement and evaluate the following methods.

- CA-GRRG and CA-GRRG⁺. The GRRG-based algorithms proposed in Section 4.
- CA-LEG. The baseline via live-edge graphs introduced in Section 4.2.
- SUBSIM [11]. The state-of-the-art IM algorithm. We use it to identify the $\lceil n/100 \rceil$ most influential nodes, set their forwarding propensity to 1, and set all others to ξ_{10} .
- $\xi \equiv \xi_{10}$. Set the forwarding propensity of every node to ξ_{10} .
- UniRand. Independently samples ξ_u for each $u \in V$ from the uniform distribution on $[\xi_{10}, 1]$.

Evaluation. The engagement under each configuration ξ is estimated using 10,000 Monte Carlo simulations. The engagement improvement achieved by a solution ξ^* is measured as the difference between its estimated engagement and that of the no-control

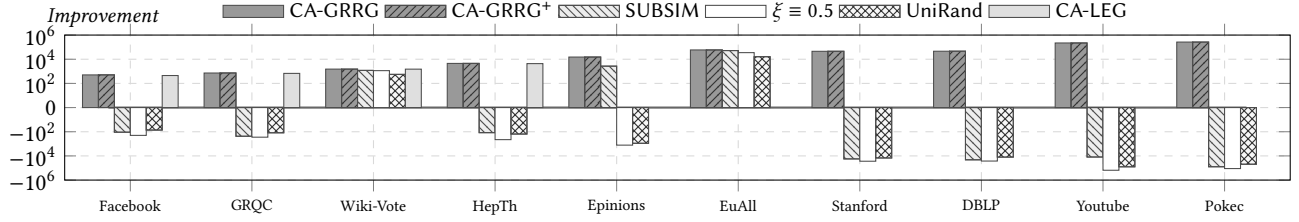


Figure 2: Engagement improvement on ten datasets.

Table 4: Runtime breakdown of three CA-based algorithms.

		Facebook	GRQC	Wiki-Vote	HepTh	Epinions	EuAll	Stanford	DBLP	Youtube	Pokec
Sampling time (s)	CA-LEG	0.86	0.55	0.68	3.15	—	—	—	—	—	—
	CA-GRRG	0.36	0.36	0.33	1.76	5.94	16.67	25.19	28.43	197.52	193.36
	CA-GRRG+	0.34	0.33	0.31	1.70	5.18	15.90	24.65	28.27	200.03	200.60
Search time (s)	CA-LEG	731.82	1,237.50	932.65	31,154.67	—	—	—	—	—	—
	CA-GRRG	3.33	2.61	1.33	13.91	37.35	56.50	186.87	224.85	877.00	1,931.33
	CA-GRRG+	1.92	1.19	0.51	6.29	13.90	22.53	90.11	117.09	383.96	882.89
Sweeps to converge	CA-LEG	2.9	3	3	3	—	—	—	—	—	—
	CA-GRRG	4.8	4.3	4	4.8	4.6	4.3	5.6	5.5	5.3	6.5
	CA-GRRG+	4.8	4	3.4	4.8	4.2	4.1	5.5	5.1	5.2	6.8

configuration $\xi = 1$. Each method is run 10 times under each experimental setting, and the average results are reported.

6.2 Results

Exp-1: Overall Performance. In Figure 2, we demonstrate the effectiveness of the proposed techniques on ten datasets under the default settings. The results show that CA-GRRG and CA-GRRG+ consistently achieve the highest engagement improvements on all datasets. The near-identical performance of the two methods confirms that the pruning procedure does not affect solution quality. CA-LEG also yields similar improvement as it also follows the CA framework. In contrast, other baselines yield substantially smaller improvements and often even decrease engagement, particularly on the larger datasets. Note that CA-LEG cannot finish within the 24-hour time limit on datasets larger than HepTh. Therefore, the corresponding values are not shown in the figure.

Table 4 reports the runtime breakdown of the three CA-based algorithms. The sampling time of CA-GRRG and CA-GRRG+ ranges from seconds to minutes and grows with graph size. The sampling time of CA-LEG is comparable on the small datasets where it completes, but its search time dominates the total cost, reaching hundreds to tens of thousands of seconds, which makes it unable to finish within 24 hours on any dataset larger than HepTh. Although CA-LEG uses fewer sweeps to converge due to a smaller sample budget (heuristically set to 1,000), each sweep of CA-LEG remains considerably more expensive. In contrast, CA-GRRG and CA-GRRG+ achieve a speedup of one to more than two orders of magnitude over CA-LEG in search time on the datasets where CA-LEG completes, without increasing sampling time. The proposed pruning technique cuts the search time of CA-GRRG+ to roughly half that of CA-GRRG on every dataset, with little effect on the number of sweeps. Both CA-GRRG and CA-GRRG+ scale to all ten datasets.

Exp-2: Varying ξ_{lo} . Figure 3 reports the engagement improvement as ξ_{lo} varies from 0.1 to 0.9. The improvement generally decreases with ξ_{lo} because a higher lower bound narrows the feasible control range. Both CA-GRRG and CA-GRRG+ consistently achieve the largest improvements across all datasets, and their nearly overlapping results confirm that the pruning technique preserves solution quality. On the four smallest datasets where CA-LEG completes, its performance is comparable to that of our methods. In contrast, other baselines yield substantially smaller improvements and often perform worse than the no-control configuration. Notably, as ξ_{lo} increases, the improvement of CA-GRRG and CA-GRRG+ decreases monotonically because the feasible range $[\xi_{lo}, 1]$ narrows, compressing the decision space and diminishing the platform’s ability to differentiate users through forwarding control. These results demonstrate that our methods remain effective under different control ranges.

Figure 4 reports the solve time under different values of ξ_{lo} . The solve times of CA-GRRG and CA-GRRG+ remain relatively stable as ξ_{lo} varies, indicating that their computational efficiency is insensitive to the control range. Moreover, CA-GRRG+ consistently outperforms CA-GRRG across all ten datasets, with more pronounced time savings on the larger datasets. On the four smallest datasets where MCG completes, both CA-GRRG and CA-GRRG+ are approximately two to three orders of magnitude faster.

Exp-3: Varying constant λ . In this experiment, we fix λ at different constants and evaluate algorithm performance. The effectiveness results are shown in Figure 5. Across all six datasets, the improvement of every method over the all-encourage baseline $\xi \equiv 1$ grows with λ , because stronger forwarding friction makes suppressing certain users increasingly beneficial. CA-GRRG and CA-GRRG+ are nearly identical and achieve the highest improvement at every value of λ . The all-suppress baseline $\xi \equiv \xi_{lo}$ and the random baseline $\xi \sim U(\xi_{lo}, 1)$ also improve with λ but remain well below the CA-based methods. As λ approaches 1, the gap between CA-GRRG

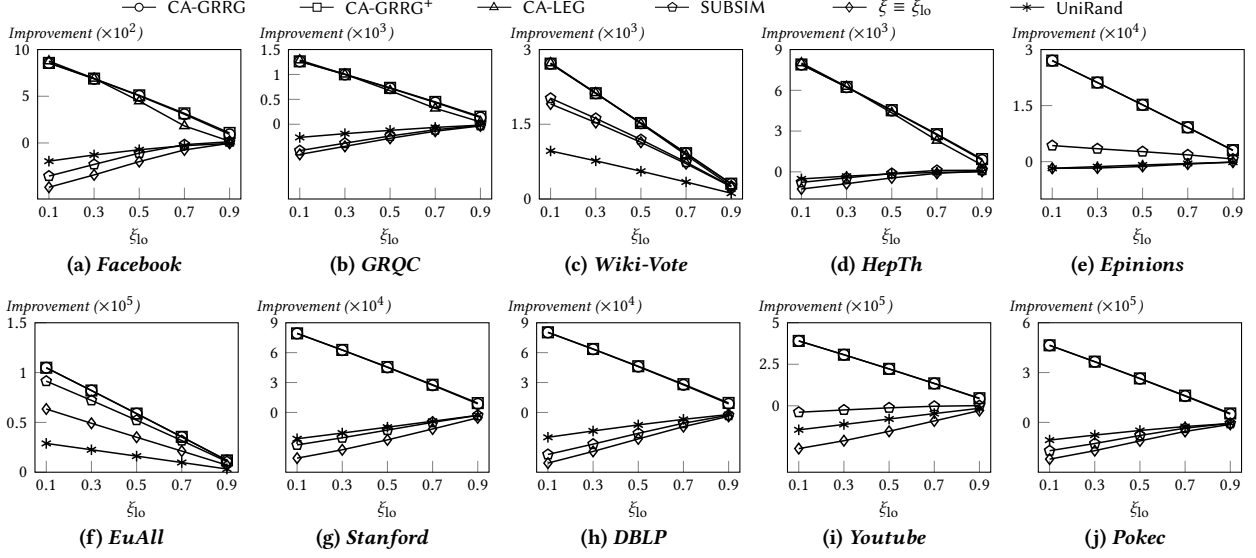


Figure 3: Improvement vs. ξ_{lo} on ten datasets.

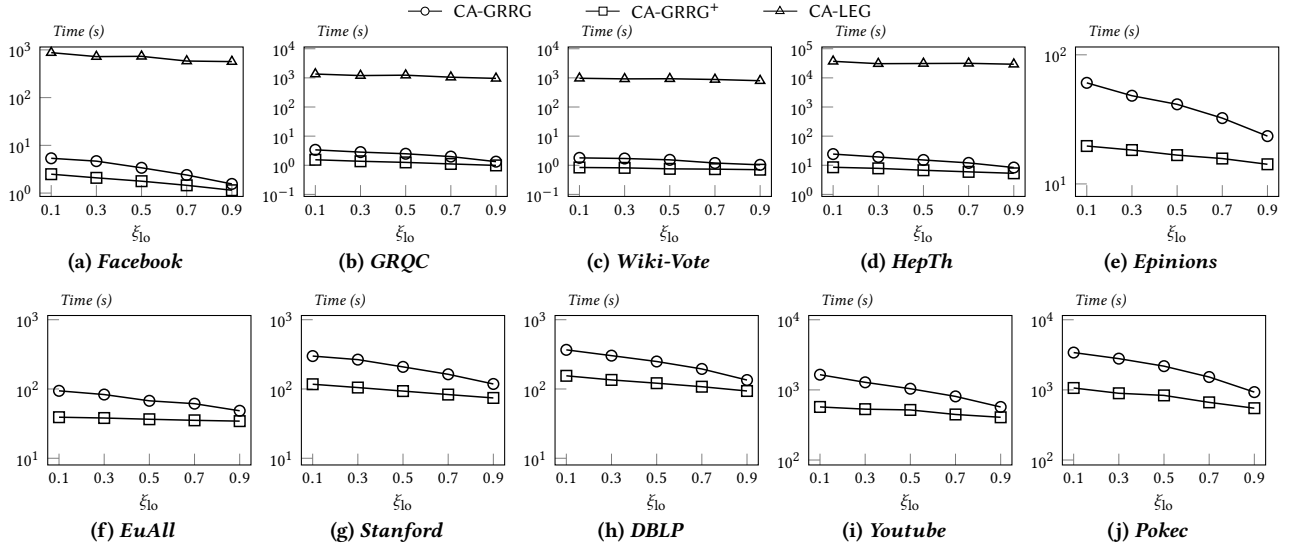


Figure 4: Solve time vs. ξ_{lo} on ten datasets.

and the simpler baselines narrows, because high friction drives the optimal strategy toward all-suppress and the correct decision becomes easy.

The running time is reported in Figure 6. CA-GRRG⁺ is faster than CA-GRRG on every dataset, with the speedup varying from roughly 1.5 $\times$ to 2.5 $\times$ depending on the dataset and the value of λ . The running time of both methods is largely insensitive to the friction level, fluctuating mildly as λ moves from 0.2 to 1.0. The absolute running time grows with graph size. The consistent gap between the two curves confirms that pruning remains effective across all friction levels.

Exp-4: Varying ε . Figure 7 shows the engagement improvement as ε varies from 0.2 to 1. Due to the page limit, we report results

on the six smallest datasets and the remaining four datasets exhibit the same trend. The improvements of CA-GRRG and CA-GRRG⁺ decrease gently as ε grows, with the value at $\varepsilon = 1$ retaining between 84% and 98% of the value at $\varepsilon = 0.2$ across the six datasets. The curves of CA-GRRG and CA-GRRG⁺ overlap almost completely under each setting. These results suggest that a moderate value of ε balances sampling cost against solution quality.

Figure 8 reports the running time for varying values of ε . The running time of both CA-GRRG and CA-GRRG⁺ decreases substantially as ε increases because a larger ε requires fewer samples. CA-GRRG⁺ is consistently faster than CA-GRRG across all datasets and parameter settings, demonstrating the effectiveness of the pruning technique in reducing computational cost. Together with Figure 7,

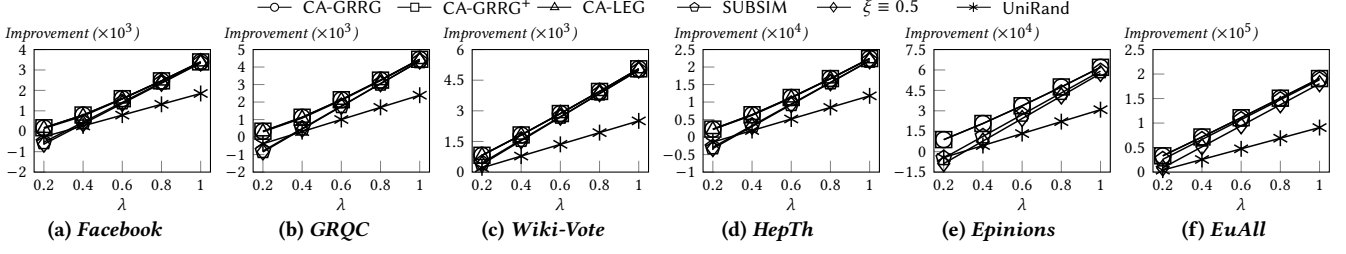


Figure 5: Improvement vs. λ on six datasets.

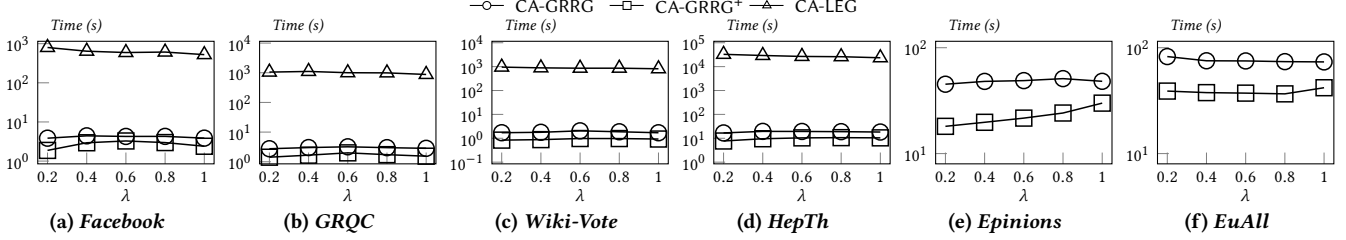


Figure 6: Solve time vs. λ on six datasets.

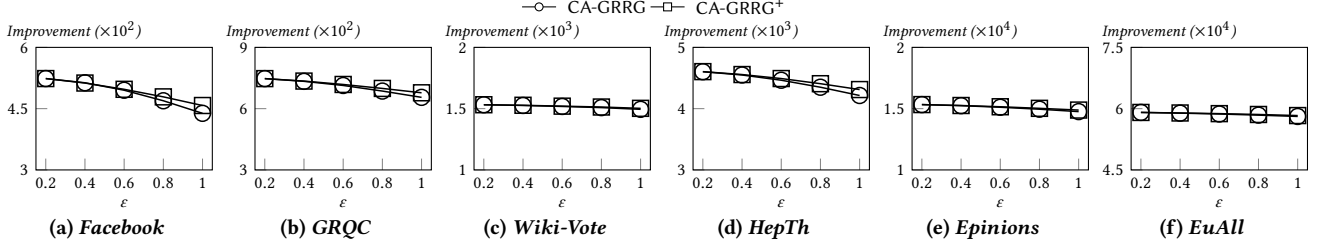


Figure 7: Improvement vs. ε on six datasets.

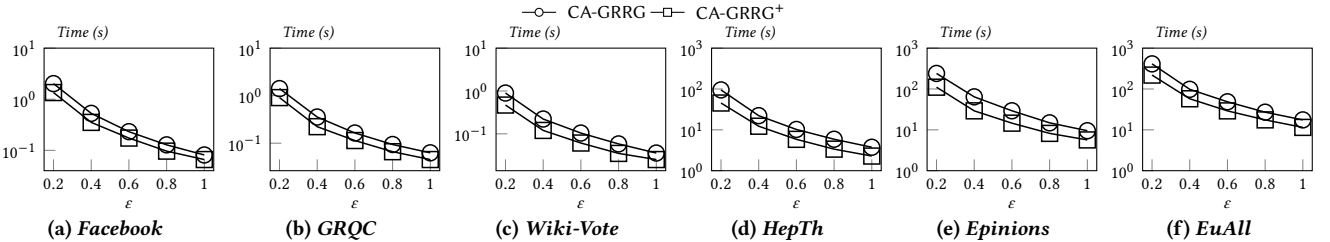


Figure 8: Solve time vs. ε on six datasets.

these results show that ε controls the tradeoff between efficacy and efficiency.

7 CONCLUSION

We propose a new *Engagement Maximization via Forwarding Control* (EMFC) problem, a cascade-control problem where the platform adjusts per-user forwarding propensities to maximize total engagement under the Independent Cascade model. We proved the objective is multilinear and bang-bang, reducing the continuous problem to a per-user binary decision, yet still inapproximable to any constant factor. To overcome the #P-hardness of evaluating

the objective, we introduced the gated reverse-reachable graph (GRRG), a sampling structure enabling unbiased estimation of the engagement under any control vector from a single set of samples. The resulting coordinate-ascent algorithm reaches an approximate coordinate-local optimum. We further accelerated the search via a pruning technique based on gradient decomposition, which locks users whose preferred endpoint is already determined before the sweep. The pruning stage yields a speedup of over 50%. Experiments on ten real-world datasets show improved engagement over natural heuristics, orders-of-magnitude speedup over a snapshot-based method at comparable quality, and scalability to million-node graphs.

REFERENCES

- [1] Christian Borgs, Michael Brautbar, Jennifer Chayes, and Brendan Lucier. 2014. Maximizing social influence in nearly optimal time. In *Proceedings of the twenty-fifth annual ACM-SIAM symposium on Discrete algorithms*. SIAM, 946–957.
- [2] Wei Chen, Chi Wang, and Yajun Wang. 2010. Scalable influence maximization for prevalent viral marketing in large-scale social networks. In *Proceedings of the 16th ACM SIGKDD international conference on Knowledge discovery and data mining*. 1029–1038.
- [3] Wei Chen, Yajun Wang, and Siyu Yang. 2009. Efficient influence maximization in social networks. In *Proceedings of the 15th ACM SIGKDD international conference on Knowledge discovery and data mining*. 199–208.
- [4] Wei Chen, Ruihan Wu, and Zheng Yu. 2020. Scalable lattice influence maximization. *IEEE Transactions on Computational Social Systems* 7, 4 (2020), 956–970.
- [5] Wei Chen, Yifei Yuan, and Li Zhang. 2010. Scalable influence maximization in social networks under the linear threshold model. In *2010 IEEE international conference on data mining*. IEEE, 88–97.
- [6] Wei Chen, Weizhong Zhang, and Haoyu Zhao. 2020. Gradient method for continuous influence maximization with budget-saving considerations. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 34. 43–50.
- [7] Suqi Cheng, Huawei Shen, Junming Huang, Wei Chen, and Xueqi Cheng. 2014. Imrank: influence maximization via finding self-consistent ranking. In *Proceedings of the 37th international ACM SIGIR conference on Research & development in information retrieval*. 475–484.
- [8] Pedro Domingos and Matt Richardson. 2001. Mining the network value of customers. In *Proceedings of the seventh ACM SIGKDD international conference on Knowledge discovery and data mining*. 57–66.
- [9] Amit Goyal, Wei Lu, and Laks VS Lakshmanan. 2011. Celf++ optimizing the greedy algorithm for influence maximization in social networks. In *Proceedings of the 20th international conference companion on World wide web*. 47–48.
- [10] Amit Goyal, Wei Lu, and Laks VS Lakshmanan. 2011. Simpath: An efficient algorithm for influence maximization under the linear threshold model. In *2011 IEEE 11th international conference on data mining*. IEEE, 211–220.
- [11] Qintian Guo, Sibao Wang, Zhewei Wei, Wenqing Lin, and Jing Tang. 2022. Influence Maximization Revisited: Efficient Sampling with Bound Tightened. *ACM Transactions on Database Systems (TODS)* 47, 3 (2022), 1–45.
- [12] Irina Heimbach and Oliver Hinz. 2018. The impact of sharing mechanism design on content sharing in online social networks. *Information Systems Research* 29, 3 (2018), 592–611.
- [13] Keke Huang, Sibao Wang, Glenn Bevilacqua, Xiaokui Xiao, and Laks VS Lakshmanan. 2017. Revisiting the stop-and-stare algorithms for influence maximization. *Proceedings of the VLDB Endowment* 10, 9 (2017), 913–924.
- [14] Rishabh Iyer and Jeff Bilmes. 2012. Algorithms for approximate minimization of the difference between submodular functions, with applications. *arXiv preprint arXiv:1207.0560* (2012).
- [15] Kyomin Jung, Wooream Heo, and Wei Chen. 2012. Irie: Scalable and robust influence maximization in social networks. In *2012 IEEE 12th international conference on data mining*. IEEE, 918–923.
- [16] David Kempe, Jon Kleinberg, and Éva Tardos. 2003. Maximizing the spread of influence through a social network. In *Proceedings of the ninth ACM SIGKDD international conference on Knowledge discovery and data mining*. 137–146.
- [17] Haewoon Kwak, Changhyun Lee, Hosung Park, and Sue Moon. 2010. What is Twitter, a social network or a news media? In *WWW '10: Proceedings of the 19th international conference on World wide web* (Raleigh, North Carolina, USA). ACM, New York, NY, USA, 591–600. <https://doi.org/10.1145/1772690.1772751>
- [18] Jure Leskovec, Andreas Krause, Carlos Guestrin, Christos Faloutsos, Jeanne VanBriesen, and Natalie Glance. 2007. Cost-effective outbreak detection in networks. In *Proceedings of the 13th ACM SIGKDD international conference on Knowledge discovery and data mining*. 420–429.
- [19] Jure Leskovec and Andrej Krevl. 2014. SNAP Datasets: Stanford Large Network Dataset Collection. <http://snap.stanford.edu/data>.
- [20] Yishi Lin, Wei Chen, and John CS Lui. 2017. Boosting information spread: An algorithmic approach. In *2017 IEEE 33rd International Conference on Data Engineering (ICDE)*. IEEE, 883–894.
- [21] Martial L Ndeffo Mbah, Jingzhou Liu, Chris T Bauch, Yonas I Tekel, Jan Medlock, Lauren Ancel Meyers, and Alison P Galvani. 2012. The impact of imitation on vaccination behavior in social contact networks. *PLoS computational biology* 8, 4 (2012), e1002469.
- [22] Hung T. Nguyen, Thang N. Dinh, and My T. Thai. 2016. Cost-aware Targeted Viral Marketing in billion-scale networks. In *IEEE INFOCOM 2016 - The 35th Annual IEEE International Conference on Computer Communications*. 1–9. <https://doi.org/10.1109/INFOCOM.2016.7524377>
- [23] Hung T. Nguyen, My T. Thai, and Thang N. Dinh. 2016. Stop-and-stare: Optimal sampling algorithms for viral marketing in billion-scale networks. In *Proceedings of the 2016 international conference on management of data*. 695–710.
- [24] Naoto Ohsaka, Takuya Akiba, Yuichi Yoshida, and Ken-ichi Kawarabayashi. 2014. Fast and accurate influence maximization on large networks with pruned monte-carlo simulations. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 28.
- [25] Jing Tang, Xueyan Tang, Xiaokui Xiao, and Junsong Yuan. 2018. Online processing algorithms for influence maximization. In *Proceedings of the 2018 International Conference on Management of Data*. 991–1005.
- [26] Youze Tang, Yanchen Shi, and Xiaokui Xiao. 2015. Influence maximization in near-linear time: A martingale approach. In *Proceedings of the 2015 ACM SIGMOD international conference on management of data*. 1539–1554.
- [27] Youze Tang, Xiaokui Xiao, and Yanchen Shi. 2014. Influence maximization: Near-optimal time complexity meets practical efficiency. In *Proceedings of the 2014 ACM SIGMOD international conference on Management of data*. 75–86.
- [28] Stephen J Wright. 2015. Coordinate descent algorithms. *Mathematical programming* 151, 1 (2015), 3–34.
- [29] Yu Yang, Xiangbo Mao, Jian Pei, and Xiaofei He. 2016. Continuous influence maximization: What discounts should we offer to social network users? In *Proceedings of the 2016 international conference on management of data*. 727–741.
- [30] Shunyu Yao, Neng Fan, and Jie Hu. 2022. Modeling the spread of infectious diseases through influence maximization. *Optimization letters* 16, 5 (2022), 1563–1586.

A MISSING PROOFS

PROOF OF THEOREM 4.2. Fix a control vector ξ . We first describe the random process generating a GRRG. Step 1: draw a root $R.r = v$ with probability τ_v/T . Step 2: from v , run a reverse BFS under $\xi \equiv 1$, retaining each incoming edge (u, v) independently with probability $p_{u,v}$, and marking each encountered node u as a q -hit with probability q_u . The reverse BFS produces a live-edge subgraph (V_R, E_R) that is identically distributed to the reverse-reachable component of v in a random live-edge graph drawn from the IC model. Step 3: prune to the forward cone from the q -hit seeds in H , keeping only nodes and edges on a directed path from some seed to the root v . Step 4: assign each non-root node u an independent gate threshold $\xi^g(u) \sim \text{Uniform}[0, 1]$, frozen thereafter.

Under a fixed ξ , the sample R is active ($I_R(\xi) = 1$) precisely when the root v is reachable from at least one q -hit seed along a path whose every non-root node u satisfies $\xi^g(u) \leq \xi_u$. Because $\xi^g(u)$ is uniform on $[0, 1]$ and independent of the graph structure, this event coincides with the root v being activated in one realization of the IC model under the control ξ , where each activated user u forwards with probability ξ_u . Formally,

$$\Pr[I_R(\xi) = 1 \mid R.r = v] = \Pr[v \in A(\xi)]. \quad (14)$$

Now expand the expectation:

$$\begin{aligned} \mathbb{E}\left[(1 - \lambda_{R.r} \xi_{R.r}) I_R(\xi)\right] &= \sum_{v \in V} \Pr[R.r = v] (1 - \lambda_v \xi_v) \Pr[I_R(\xi) = 1 \mid R.r = v] \\ &= \sum_{v \in V} \frac{\tau_v}{T} (1 - \lambda_v \xi_v) \Pr[v \in A(\xi)] \quad (\text{by (14)}) \\ &= \frac{1}{T} J(\xi) \quad (\text{by (1)}). \end{aligned}$$

Multiplying both sides by T yields the claim. $\square$

PROOF OF THEOREM 4.5. The proof follows [27]. $\square$

B ADDITION EVALUATION

Performance on different distribution of q . Figure 9 reports the improvement when seed probabilities are drawn uniformly from $[0, 1]$ instead of the main-experiment setting. CA-GRRG and CA-GRRG⁺ remain the best performers across all ten datasets and produce nearly identical results, indicating that pruning incurs no quality loss under this distribution either. CA-LEG matches CA-GRRG on the four smallest graphs but fails to scale beyond them. SubSim trails slightly, and the two fixed-control heuristics ($\xi \equiv$

0.5 and Uniform-Random) deteriorate sharply, yielding negative improvements on larger graphs where indiscriminate suppression destroys reach. Compared to the main experiment, the overall gains are smaller, since uniformly random q weakens the correlation between seed placement and session value that CA-GRRG and CA-GRRG⁺ exploit through value-weighted root sampling.

Performance on different distribution of λ . Figure 10 replaces the per-user friction coefficients with an exponentially distributed

λ , which concentrates more users toward low friction and fewer toward high friction. The ranking of methods is consistent with earlier results: CA-GRRG and CA-GRRG⁺ lead, SUBSIM follows closely, and the fixed heuristics remain behind. No method suffers from negative improvement in this setting, suggesting that the skewed λ distribution makes the suppression-versus-encourage decision less brittle than under other distributions. CA-LEG again fails on the six largest datasets due to its per-flip forward propagation cost.

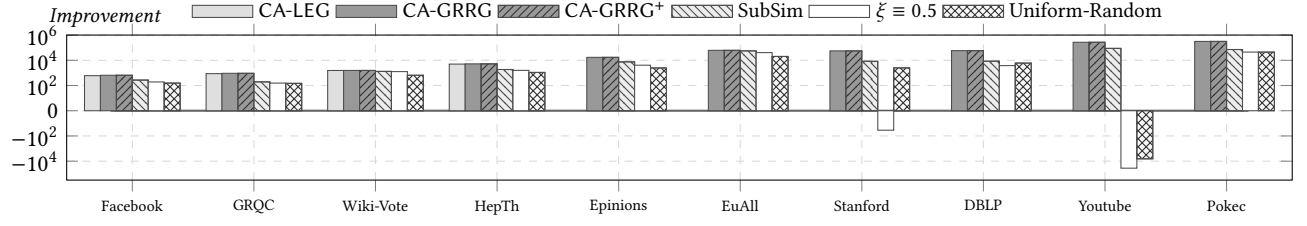


Figure 9: Improvement (method – $\xi \equiv 1$) with $q \sim \text{Uniform}[0, 1]$ ($\varepsilon = 1$, 10-seed average). τ and λ match the main experiment.

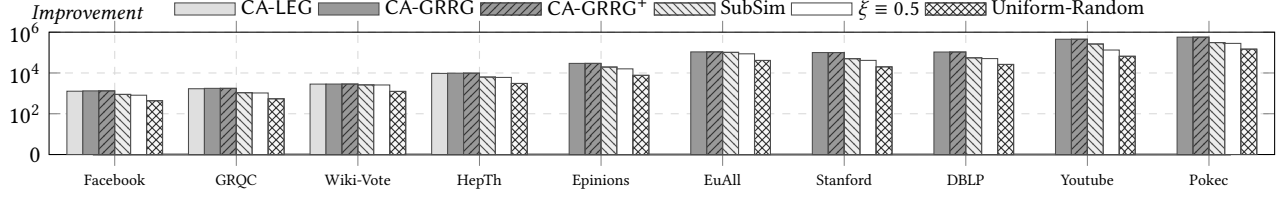


Figure 10: Improvement (method – $\xi \equiv 1$) on ten datasets ($\lambda \sim \text{TruncExp}[0, 1]$, $\varepsilon = 1$, 10-seed average).